

A Resilient and Decoupled Cloud-Native Serverless Architecture Deployed on AWS

1. Introduction

The evolution of Cloud Computing has transitioned infrastructure management from traditional IaaS models to Serverless and Function-as-a-Service (FaaS) paradigms. While native serverless architectures provide out-of-the-box scalability and high availability, relying solely on default cloud provider behaviors often leads to uncontrolled scaling, potential downstream bottlenecks, and a lack of application-level fault tolerance.

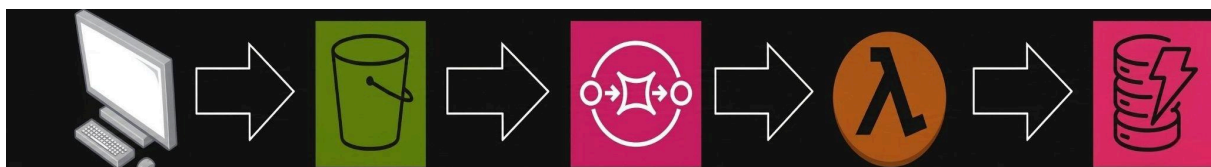
This project aims to implement an enterprise-grade, 100% serverless, event-driven data processing pipeline on Amazon Web Services (AWS). Building upon foundational cloud concepts, this advanced architecture introduces strict asynchronous decoupling, controlled compute scaling, and robust error handling. The objective is to practically demonstrate advanced Cloud Computing concepts, specifically addressing: Controlled Rapid Elasticity, Fault Tolerance via Dead Letter Queues (DLQ), Service Decoupling via Message Brokers, and strict Security Governance through the Least Privilege principle.

2. System Architecture

2.1 Overview

The revised architecture shifts from a simple reactive pipeline to a fully decoupled, message-driven system designed to handle massive load spikes without overwhelming downstream resources.

Architecture Flow: Local Python Script (Massive Load) → Amazon S3 (Data Lake) → Amazon SQS (Message Broker / Buffer) → AWS Lambda (Compute Layer - Rate Limited) → Amazon DynamoDB (State Persistence) (Branching from SQS) → SQS Dead Letter Queue (Fault Tolerance for corrupted data)



The system simulates a massive IoT workload. A local Python script generates hundreds of concurrent JSON payloads simulating sensor telemetry. Upon ingestion into S3, an event notification is pushed to an SQS queue rather than invoking the compute layer directly. The queue acts as a shock absorber. AWS Lambda consumes messages from the queue with a strictly configured maximum concurrency, processing the business logic and persisting the results in DynamoDB. Unprocessable or corrupted files are automatically isolated in a Dead Letter Queue.

2.2 Core Components

- Amazon S3 (Data Lake): Acts as the ingestion point. It triggers an `s3:ObjectCreated:*` event notification routed directly to SQS.
- Amazon SQS (Buffer & Decoupling): The IoT-Main-Queue completely decouples storage from compute. It absorbs sudden traffic spikes, holding the messages until the compute layer is ready to process them, preventing system overloads.
- AWS Lambda (Compute Layer): A stateless Python microservice. It retrieves the file from S3, parses the JSON payload, executes conditional business logic (classifying temperature data into NORMAL, WARNING, or CRITICAL_ALARM), and prepares the database record.
- Amazon SQS DLQ (Fault Tolerance): The IoT-DLQ captures messages that Lambda fails to process after a configured number of retries, ensuring the main queue is not blocked by "poison pill" messages.
- Amazon DynamoDB: A managed NoSQL database acting as the persistence layer for the processed IoT metadata.
- AWS IAM: Handles Identity and Access Management through strictly scoped inline JSON policies.

3. Implementation Details and Advanced Configuration

3.1 Event-Driven Decoupling

Unlike basic architectures where S3 directly invokes a Lambda function (which can lead to thousands of concurrent executions and potential database throttling), this architecture implements the Queue-Based Load Leveling pattern. S3 sends lightweight notifications to SQS. Lambda is configured with an Event Source Mapping to poll the queue asynchronously.

3.2 Advanced Compute Logic

The Lambda function was upgraded to perform real data processing rather than simple metadata extraction. Upon receiving the SQS message, the function:

1. Downloads the actual payload from S3 using boto3.
2. Parses the JSON structure (validating its integrity).
3. Evaluates the temperature parameter against predefined thresholds.
4. Generates a unique message_id (UUID) and a timestamp before persisting the enriched item into DynamoDB.

4. Non-Functional Properties

4.1 Controlled Elasticity vs. Unbounded Scaling

A critical flaw in naive serverless implementations is allowing functions to scale to the account's maximum concurrency (default 1,000). While this demonstrates Rapid Elasticity, it often results in severe throttling on downstream databases (like DynamoDB in provisioned mode) and unexpected cost spikes. To demonstrate architectural control, the Lambda trigger was configured with a Maximum Concurrency of 5. Regardless of how many thousands of objects land in S3 simultaneously, AWS will only provision a maximum of 5 concurrent Lambda environments. The SQS queue absorbs the excess load, dynamically buffering the requests. This proves that scaling is explicitly architected and controlled by the developer, not blindly delegated to the cloud provider.

4.2 High Availability and Fault Tolerance (Dead Letter Queue)

Relying entirely on AWS infrastructure availability is insufficient for enterprise fault tolerance; application-level errors must be handled. To guarantee zero data loss and uninterrupted processing, a Dead Letter Queue (DLQ) was implemented. The main SQS queue is configured with a Maximum Receives policy of 2. If the Lambda function encounters a fatal error (e.g., attempting to parse a corrupted, non-JSON file), the function crashes, and the message is returned to the queue. After two failed attempts, SQS automatically routes the "poisonous" message to the DLQ. This allows the system to continue processing healthy files while preserving corrupted data for future debugging.

4.3 Security and Strict Least Privilege

Security and Governance were enforced bypassing AWS Managed Policies in favor of custom, highly restrictive inline JSON policies. The IAM Role assumed by the Lambda function contains explicitly defined Resources and Actions:

- sqs:ReceiveMessage and sqs:DeleteMessage restricted only to the ARN of the IoT-Main-Queue.
- s3:GetObject restricted only to the specific testing bucket.
- dynamodb:PutItem restricted only to the IoT_Telemetry table. Any attempt to access unauthorized resources or perform unlisted actions is actively blocked by the AWS authorization engine.

Least_Privilege

☐	Nome della policy ↗	▲	Tipo	▼	E
☐	StrictPolicy		Customer inline		0

StrictPolicy

```

1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Allow",
6        "Action": [
7          "sqs:ReceiveMessage",
8          "sqs:DeleteMessage",
9          "sqs:GetQueueAttributes"
10       ],
11       "Resource": "arn:aws:sqs:eu-south-1:886166401501:IoT-Main-Queue"
12     },
13     {
14       "Effect": "Allow",
15       "Action": [
16         "s3:GetObject"
17       ],
18       "Resource": "arn:aws:s3:::matteo-iot-stress-test-2024/*"
19     },
20   ]
  
```

5. Demonstration and Load Testing Results

To empirically validate the architecture, a highly concurrent Python stress-test script (ThreadPoolExecutor with 20 workers) was executed from a local environment.

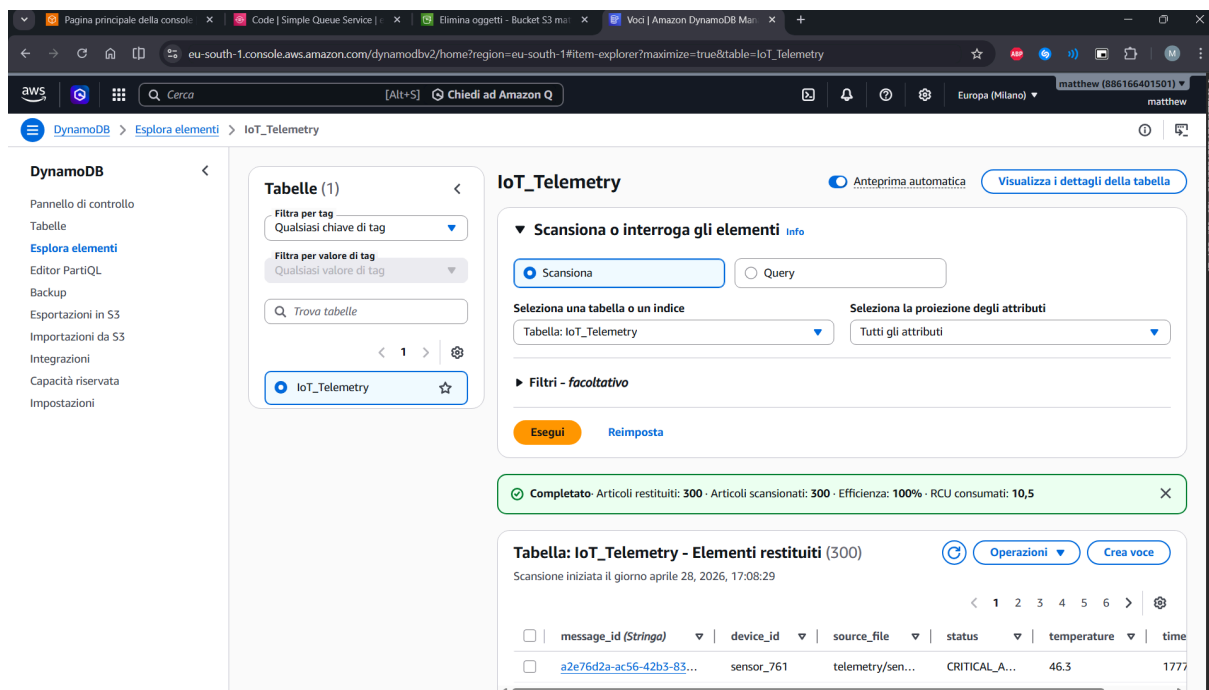
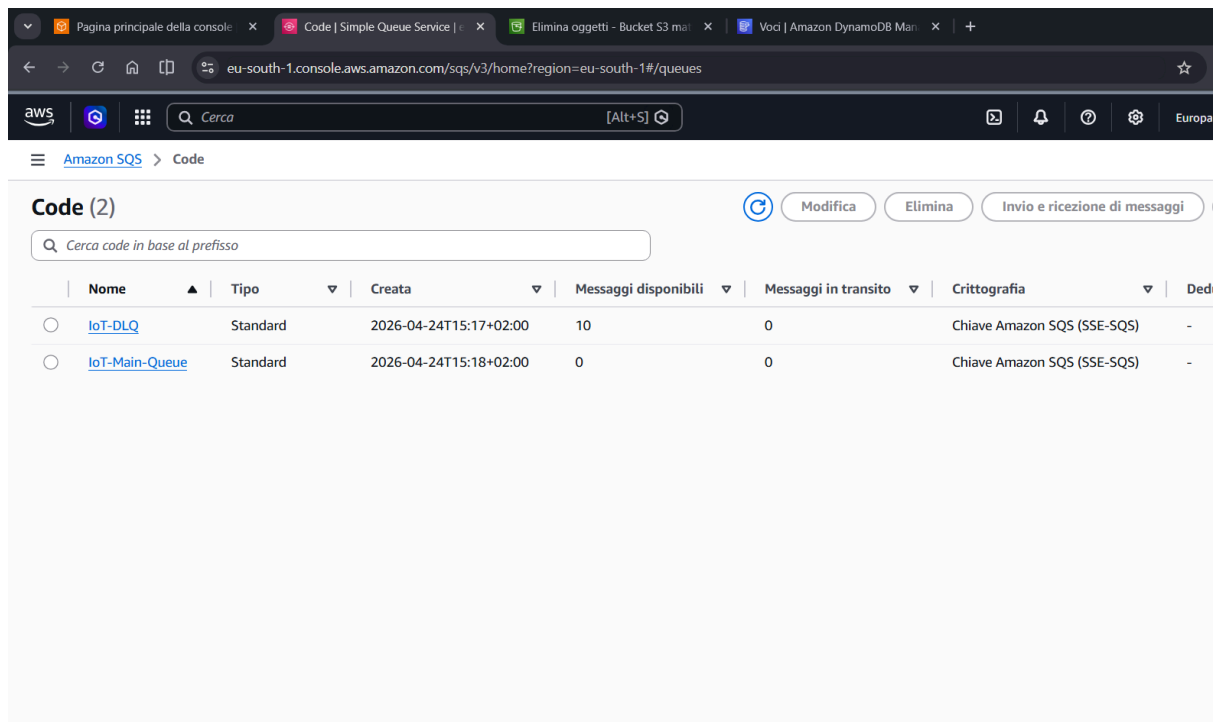
Test Scenario: The script was programmed to inject a massive, sudden load into the S3 bucket:

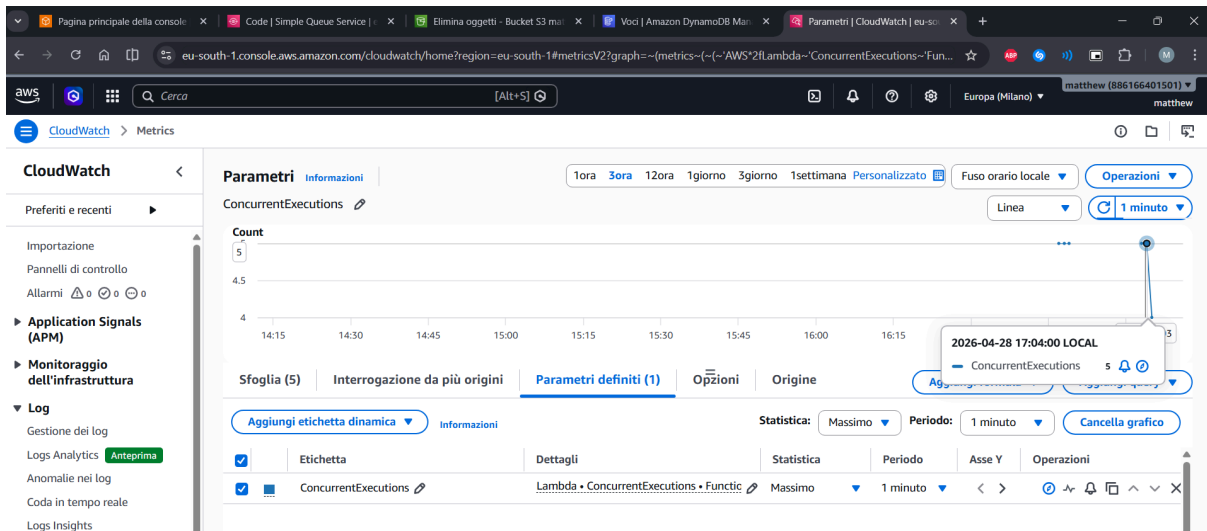
- 300 valid JSON files simulating correct IoT telemetry.
- 10 corrupted files (plain text instead of JSON) injected randomly into the payload batch.

Observability and Results:

1. Load Absorption (SQS): Immediately after execution, the S3 dashboard showed 310 files ingested. The SQS dashboard visually demonstrated the buffering effect: the "Messages Available" metric spiked to ~310.

2. Controlled Processing (Lambda): Instead of processing all files in 1 second, CloudWatch metrics confirmed that the ConcurrentExecutions never exceeded 5. The queue depth decreased steadily over approximately 60 seconds, proving the effectiveness of the rate-limiting configuration.
3. Fault Tolerance Validation (DLQ): Once the main queue was emptied, the DynamoDB table contained exactly 300 perfectly processed records (categorized into NORMAL, WARNING, and CRITICAL_ALARM). Simultaneously, the SQS IoT-DLQ showed exactly 10 messages available. The system successfully caught the 10 corrupted files, isolated them, and prevented a total pipeline failure without any manual intervention.





6. Conclusion

This project successfully demonstrated the practical implementation of an advanced Cloud-Native Serverless architecture. By evolving from a basic reactive model to an event-driven, decoupled system utilizing Amazon SQS, the architecture achieved a production-ready level of resilience.

The implementation specifically proved that true Cloud Engineering involves actively managing cloud resources rather than passively accepting default behaviors. Implementing Maximum Concurrency limits protected downstream systems (demonstrating controlled scalability), while the integration of a Dead Letter Queue provided robust application-level Fault Tolerance. Finally, the strict application of Custom IAM roles validated the importance of Security and Governance in cloud environments.